

Methodology of Handwritten Mathematical Symbol Classification

1. Data Collection and Understanding

The dataset consists of image data accompanied by metadata and structured information in CSV files. Three primary files are loaded:

metaData.csv: Contains additional information such as image properties, labels, or metadata relevant to training.

train.csv: Contains labeled image data used to train the deep learning model.

test.csv: Includes unlabeled image data, on which the trained model is used to make predictions.

The dataset is analyzed using Pandas to inspect column names, check for missing values, and understand the distribution of classes .

Understanding the dataset's structure helps define preprocessing and model training strategies.

2. Library and Framework Selection

To build a robust and scalable image classification model, the following libraries and frameworks are used:

Deep Learning Frameworks

PyTorch: A flexible deep learning framework used to build, train, and evaluate the neural network model.

Torchvision: Provides pre-trained deep learning models dataset utilities, and image transformation functions.

Data Handling & Analysis

Pandas: Used to read and manipulate structured data from CSV files.

NumPy: Facilitates numerical operations, including matrix computations and array manipulations.

Image Processing

Transformers Enables the use of transformer-based models such as Vision Transformer

PIL (Python Imaging Library): Handles image loading and processing.

Torchvision.transforms: Applies various transformations to enhance and standardize input images.

Visualization

Matplotlib & Seaborn: Used to generate graphs for analyzing dataset distribution, loss trends, and performance metrics.

3. Data Preprocessing and Augmentation

Before training the model, the raw dataset is processed to ensure consistency and improve generalization.

Data Cleaning

Handling missing values (if any) in train.csv and metaData.csv.

Removing duplicate entries to ensure no redundant training samples.

Ensuring correct data types (e.g., converting categorical labels to numeric values).

Image Processing & Augmentation

To prepare images for input into the deep learning model, the following transformations are applied:

Resizing: Ensures that all images have a fixed dimension required by the deep learning model (e.g., 224×224 pixels for Vision Transformer or ResNet).

Normalization: Standardizes pixel values to a defined range (e.g., mean = 0.5, standard deviation = 0.5) to facilitate stable training.

Random Augmentations: Introduces variations in training data to improve model generalization:

Rotation & Flipping: Helps the model learn orientation-invariant features.

Brightness & Contrast Adjustments: Reduces sensitivity to lighting conditions.

Random Cropping & Scaling: Simulates different viewing perspectives.

These transformations are implemented using `torchvision.transforms` and applied dynamically during model training.

DataLoader for Efficient Processing

The dataset is converted into PyTorch tensors.

`DataLoader` is used to create batches of data for efficient processing on GPUs.

Mini-batch sizes are defined to balance memory usage and training efficiency.

4. Model Selection and Architecture

The deep learning model is chosen based on its suitability for image classification tasks.

Convolutional Neural Networks (CNNs)

Alternative approach using ResNet/EfficientNet from Torchvision.

CNNs use hierarchical feature extraction through convolutional layers, making them effective for spatial pattern recognition.

Final Model Architecture

The model consists of:

Feature Extraction Layer (CNN backbone): Extracts high-level features from images.

Fully Connected Layers: Maps extracted features to output labels.

Activation Functions:

ReLU (Rectified Linear Unit) for non-linearity.

Softmax for probability-based classification.

The model is built using PyTorch's `torch.nn` module, ensuring modularity and efficiency.

5. Model Training and Optimization

To train the model effectively, the following strategies are employed:

Loss Function

Cross-Entropy Loss: Used for multi-class classification.

This function calculates the error between the predicted probability distribution and actual class labels.

Optimizer Selection

Adam Optimizer: Provides adaptive learning rates for efficient weight updates.

Learning Rate Scheduling: Adjusts learning rate dynamically to avoid stagnation.

Training Process

Forward Propagation: Inputs are passed through the model to generate predictions.

Loss Calculation: Compares predictions to actual labels.

Backpropagation: Computes gradients using PyTorch's automatic differentiation.

Weight Updates: The optimizer updates model parameters to minimize loss.

The model is trained over multiple epochs, with validation data used to monitor performance.

6. Model Evaluation and Performance Analysis

To assess model performance, various evaluation metrics are computed.

Accuracy and Loss Tracking

Training and validation accuracy/loss are plotted to observe model convergence.

If overfitting occurs (validation loss increases while training loss decreases), regularization techniques (dropout, weight decay) are applied.

Confusion Matrix Analysis

A confusion matrix is generated to identify misclassifications.

Helps in analyzing class-wise model performance.

Precision, Recall, and F1-Score

Precision: Measures how many predicted positive instances are correct.

Recall: Measures how many actual positive instances are identified correctly.

F1-Score: Harmonic mean of precision and recall, ensuring balanced evaluation.

The model is fine-tuned based on these performance indicators.

Here are some of the modifications and methodologies we implemented throughout the process to refine and optimize the model, ensuring the best possible performance.

Multiple Optimizers Tested: Various optimizers, including SGD, Adam, and RMSprop, were evaluated to compare their effects on convergence speed and stability. Adam was found to perform well due to its adaptive learning rate adjustments.

Learning Rate Experiments: Different learning rates, ranging from $1e-1$ to $1e-6$, were tested to find an optimal value that ensures both fast convergence and stability. Lower learning rates improved fine-tuning, while higher rates accelerated initial learning but risked instability.

Scheduler Implementation: Various learning rate schedulers, such as StepLR, ReduceLROnPlateau, were applied to dynamically adjust the learning rate. StepLR helped maintain stable progress, while ReduceLROnPlateau adjusted learning rates based on validation loss trends, preventing stagnation.

Impact on Model Performance: These optimization strategies significantly influenced training efficiency and final accuracy. The right combination of optimizer, learning rate, and scheduler led to improved generalization, reduced overfitting, and enhanced performance on unseen data.

7. Final Model Selection and Inference

The best-performing model (ViT or CNN-based) is chosen based on validation accuracy.

The final model is tested on unseen data from the test.csv dataset.

Predictions are stored in a structured format for submission.

8. Model Deployment and Submission

Once the model achieves the desired accuracy, it is used to generate predictions on test images.

Predictions are converted into a CSV file, formatted according to competition requirements.

The model is saved using PyTorch's `torch.save()` function, enabling future deployment or fine-tuning.

Conclusion

This methodology follows a structured approach to building a high-performing deep learning model for image classification. The integration of transfer learning, data augmentation, and performance analysis ensures optimal model accuracy. The step-by-step refinement process, from preprocessing to inference, maximizes the model's generalization ability, making it well-suited for the competition task.

CB.SC.U4CSE23245 – SHAIK ABDUS SATTAR

CB.SC.U4CSE23206 - A C SANHITHA REDDY